Practical Multilayer Network Analysis with Py3plex

Release 2.0.92

Blaž Škrlj

Nov 07, 2026

CONTENTS

1 Front Matter 1

2 Introduction: Why Multilayer Modeling Changes Results 3

3 Multilayer Basics: Semantics Before Computation 5

4 Design of py3plex: Trade-offs and Boundaries 8

5 Installation and First Verified Run 11

6 Data Loading and Representation Choices 13

7 Visualization as Diagnostic Analysis 16

8 Core Algorithms: What Is Estimated, What Is Approximated 18

9 DSL Fundamentals: A Query Mental Model 21

10 Builder API and Explain Plans: Reading Your Own Query 23

11 Advanced Workflows: Compositional Queries Under Uncertainty 26

12 Limitations and Stability: What py3plex Does Not Promise 29

13 Case Study 1 — Social Multiplex: Influence vs Brokerage 31

14 Case Study 2 — Biological Multilayer: Robust Signal or Layer Artifact? 33

15 Case Study 3 — Transportation Network: Resilience Under Layer Disruption 35

16 Testing and Validation as Methodological Controls 37

17 Reproducible Environments and Replayable Analysis 39

18 GUI Overview: Where It Helps, Where It Does Not 41

19 Appendix A: Repository Layout and Scripts 43

20 Appendix B: Docker, Docker Compose, and Deployment 45

21 Appendix C: Detailed Validation Scripts 47

22 Appendix D: Error Handling and Exception Hierarchy 49

23 Appendix E: Extended API and DSL Reference **51**

24 Bibliography **53**

Bibliography **56**

CHAPTER

ONE

FRONT MATTER

1.1 About This Book

Practical Multilayer Network Analysis with Py3plex is a technical handbook for readers who already know basic graph analytics and want to make better modeling decisions on multilayer data.

This is not a complete API catalog. It is a working text about representation, inference, failure modes, and reproducibility.

1.2 Scope and Reader Contract

The core narrative is organized around four questions:

1. When does multilayer modeling change the scientific conclusion?
2. Which py3plex implementation choices are consequential for that conclusion?
3. Which approximations are acceptable under realistic constraints?
4. How do we make the workflow auditable and reproducible?

The book assumes intermediate Python fluency, familiarity with standard graph concepts, and comfort reading short mathematical arguments.

1.3 How to Read This Book

Read the book as a staged argument. **Part I (Foundations)** defines semantics and design boundaries; **Part II (Working Practice)** turns those choices into onboarding, representation, visualization, and algorithm decisions; **Part III (DSL)** makes query logic auditable; **Part IV (Case Studies)** shows where conclusions materially change; and **Part V (Systems)** treats testing and reproducibility as scientific controls. The core chapters carry the argument, while the appendices remain software reference material for implementation detail and lookup.

1.4 What This Book Deliberately Does Not Do

- It does not claim that multilayer modeling is always superior.
- It does not treat convenience interfaces as methodological guarantees.
- It does not equate reproducible code execution with valid scientific inference.

1.5 About the Software

This edition targets **py3plex 2.0.92**. py3plex implements multilayer data structures, algorithm wrappers, DSL tooling, and workflow utilities. Some operations delegate to single-layer backends after explicit transformations; those transitions are identified in the relevant chapters.

1.6 Conventions

- Code blocks are minimal and intended to expose analytical decisions.
- We label whether a statement is a **concept**, **implementation choice**, **approximation**, or **workflow recommendation**.
- Caveats are presented inline where methods are introduced.

1.7 Acknowledgment and Citation

If py3plex contributes to published work, cite the primary toolkit paper:

Skrlj, B., Kralj, J., and Lavrac, N. (2019). *Py3plex toolkit for visualization and analysis of multilayer networks*. Applied Network Science, 4(1), 94. DOI: `10.1007/s41109-019-0203-7`.

1.8 License

The py3plex library is released under the MIT License; this book text is released under CC BY 4.0.

1.9 Running Example Used Throughout

Across Parts I–V, we repeatedly revisit a three-layer commuter network (metro, bus, and walking-transfer links) to compare a flattened ranking against a multilayer ranking, then stress-test that difference under uncertainty and disruption scenarios.

Note

Version Information: This book edition is version 2.0.92 (2026), aligned with py3plex 2.0.92 and Python 3.8+ runtime support.

CHAPTER

TWO

INTRODUCTION: WHY MULTILAYER MODELING CHANGES RESULTS

Most network tutorials begin with capabilities. This chapter begins with failure: what single-layer analysis gets wrong when relationships are heterogeneous.

2.1 Problem Framing

In many domains, edges carry incompatible semantics. Consider a hospital contact network where one layer records formal care-team coordination, one records informal consultation, and one records shared-shift overlap. Flattening those layers into one tie type mixes institutional structure, social behavior, and scheduling artifacts.

Flattening these into one graph is not just information loss. It changes the estimand. A centrality score in a flattened graph answers a different question than a score on a structured multilayer representation, and mathematically replaces layer-conditioned adjacency operators with a single aggregate operator.

2.2 A Running Contrast: Flattened vs Multilayer

Consider a commuter system with rail and bus layers:

- In a flattened graph, high transfer stations can dominate centrality by construction.
- In a multilayer model, we can separate within-mode influence from transfer dependence.

The practical consequence is interpretive: "important node" may mean resilient hub, fragile bottleneck, or mode-bridging artifact depending on representation.

Concrete estimand shift: in a flattened commuter graph, Station X can rank #2 by aggregate betweenness because all transfer traffic is merged, while in the multilayer representation it can drop below the top 20 in metro-only betweenness and rise to #1 in cross-layer brokerage. The metric name is similar, but the estimand is not.

2.3 What py3plex Contributes (and What It Does Not)

py3plex provides a multilayer data model and analysis interfaces that make these distinctions executable in Python workflows.

It does **not** remove modeling judgment. You still choose:

- how layers are defined,
- what inter-layer edges mean,
- whether approximation is acceptable,
- which uncertainty quantification method is relevant.

These choices are methodological, not merely software settings.

2.4 Three Questions to Carry Through the Book

1. **Representation:** Is a layer boundary meaningful, or only convenient?
2. **Inference:** Does an algorithm estimate what we think it estimates under this representation?
3. **Credibility:** Are the results stable under perturbation, alternative parameterizations, and reproducibility constraints?

2.5 Audience and Prerequisites

This text is aimed at graduate researchers and practitioners who already use graph analytics and now need multilayer rigor. We assume:

- Python proficiency,
- familiarity with basic graph metrics,
- willingness to inspect assumptions rather than accept defaults.

2.6 Chapter Roadmap

- Chapter 3 formalizes multilayer semantics and common traps.
- Chapter 4 explains py3plex design trade-offs.
- Part II moves to practical workflows.
- Part III focuses on DSL reasoning.
- Part IV turns methods into domain arguments through case studies.

The next chapter provides the semantic rules that make these distinctions operational rather than rhetorical.

CHAPTER

THREE

MULTILAYER BASICS: SEMANTICS BEFORE COMPUTATION

This chapter defines multilayer network objects and highlights semantic mistakes that produce misleading analyses.

3.1 Formal Object

A multilayer network can be represented as $\mathcal{M} = (V, L, V_M, E_{intra}, E_{inter})$, where:

- V is the set of physical entities,
- L is the set of layers,
- $V_M \subseteq V \times L$ is the set of node replicas,
- E_{intra} links replicas within a layer,
- E_{inter} links replicas across layers.

3.2 Concept vs Implementation

Concept: node replicas are analytical units that encode context.

py3plex implementation: nodes are stored as `(node, layer)` pairs in the core graph, so many operations naturally return replica-level results.

Do not silently reinterpret replica counts as physical-node counts.

3.3 Replica-Level vs Physical-Node-Level Reading

3.4 Replica Semantics: The First Major Pitfall

If a physical node appears in three layers, it has three replicas. This matters for:

- counts,
- degree interpretation,
- top-k selection,
- community assignments.

A frequent novice error is to compute top hubs globally, then report them as physical entities without de-duplicating by base node.

3.5 Worked Micro-Example: One Person, Different Rankings

Suppose `A` appears in three layers: social, work, and support. In social, `A` is peripheral; in work, `A` is a bridge; in support, `A` is isolated. A global top-k on aggregate degree can still elevate `A` because work-layer bridging dominates, while a per-layer ranking reveals that `A` is only locally critical in one context. The same identifier is stable, but the analytical role is not.

3.6 Degree Is Ambiguous in Multilayer Contexts

At least three notions of degree can be relevant:

- **intra-layer degree** (within one layer),
- **inter-layer degree** (coupling/transfer links),
- **aggregate degree** (combined).

In py3plex workflows, aggregate degree is often the default unless per-layer operations are explicitly applied.

Interpretive warning: aggregate degree is not necessarily a better statistic; it is a different statistic.

Wrong conclusion / corrected conclusion

Wrong conclusion: "Node B is universally the top hub because it has the highest degree."

Corrected conclusion: "Node B has the highest aggregate degree under this representation; per-layer degree shows it is dominant only in the collaboration layer and ordinary elsewhere."

3.7 Coverage Semantics and False Certainty

Coverage filters encode cross-group logic:

- `all`: intersection,
- `any`: union,
- thresholded modes (`at_least`, `fraction`): partial overlap.

`all` is strict and can yield very small result sets. This is often interpreted as "only a few robust nodes," but it may instead reflect harsh filtering under high layer heterogeneity.

3.8 Global vs Per-Layer Operations

Two analyses can be correct yet answer different questions:

- **global** community detection: seeks communities spanning layers,
- **per-layer** detection: compares community structure between layers.

Neither is universally preferred. Choose based on hypothesis, not convenience.

3.9 Minimal Sanity Checklist

Before trusting any multilayer output:

1. Confirm whether reported units are replicas or physical nodes.
2. State which degree semantics are used.

3. State whether operations were global or grouped by layer.
4. Report coverage mode and thresholds.
5. Provide at least one alternative analysis path as a robustness check (for example, rerun rankings with stratified perturbation UQ and compare top-k stability).

3.10 Why This Chapter Matters

Most downstream errors are not coding errors. They are semantic errors introduced before algorithm choice. Getting these distinctions right is the main precondition for meaningful multilayer inference.

CHAPTER

FOUR

DESIGN OF PY3PLEX: TRADE-OFFS AND BOUNDARIES

This chapter explains why py3plex is structured the way it is, and what those choices imply for analysis quality.

4.1 Design Goal

py3plex is designed to make multilayer analysis practical in Python without hiding key modeling decisions.

The project prioritizes:

1. a consistent multilayer representation,
2. interoperability with established graph tooling,
3. explicit caveats around approximations and feature maturity.

4.2 Core Architectural Choice

py3plex builds on NetworkX-compatible graph objects while layering multilayer semantics on top.

Benefit: immediate access to mature single-layer algorithms and ecosystem tooling.

Cost: some multilayer operations necessarily pass through reductions (for example, layer-restricted or projected computations). Those transitions can alter interpretation if not documented.

Concrete delegated/projection path: a workflow may start from a multilayer object, restrict to selected layers, project to a monoplex graph for a delegated NetworkX routine, then map results back to multilayer identifiers. That path is often computationally practical, but it is analytically different from a native multilayer operator.

4.3 Current Repository Surface

The repository has grown beyond the original `core/algorithms/dsl` split. The current architecture separates reusable infrastructure from analysis-facing entry points:

- `py3plex.dsl` remains the main workflow interface, with `dsl/lint` for static diagnostics and `dsl/program` for typed, rewritable `GraphProgram` objects.
- `py3plex.algebra` and `py3plex.semiring` hold algebraic path and closure machinery that supports semiring-style reasoning rather than ad-hoc shortest path special cases.
- `py3plex.embeddings` and `py3plex.ml.embedding` expose embedding primitives such as NetMF, MetaPath2Vec, Node2Vec, DeepWalk, LINE, and multiplex variants.
- `py3plex.experiments` and `py3plex.meta` turn reproducibility into explicit artifacts: stored experiment records and fixed/random-effect meta-analytic summaries across networks.

- `py3plex.out_of_core` and `py3plex.optimizer` address larger workflows through streaming execution and cost-aware planning.

These additions do not change the central design principle of the book: prefer auditable workflows that make representation, approximation, and randomness explicit.

4.4 Theory, Implementation, Workflow

Throughout py3plex, it helps to separate three categories:

- **Theory:** what a multilayer statistic means mathematically.
- **Implementation:** how py3plex computes or delegates that statistic.
- **Workflow:** how analysts sequence filtering, computation, and validation in practice.

Confusing these categories leads to overclaiming. An implementation convenience is not theoretical justification.

Native multilayer workflows are preferred when the estimand depends on layer identity (for example, cross-layer brokerage, layer coverage, or multilayer community structure). Delegated single-layer backends are often acceptable for exploratory baselines, coarse screening, or compatibility checks, provided the projection step is explicitly reported.

4.5 What Users Most Often Overclaim from Implementation Convenience

Three overclaims recur in practice:

1. "This centrality is multilayer-native" when the computation was actually run on a projection.
2. "This partition is robust across contexts" when only one delegated backend and one parameter setting were used.
3. "This ranking is stable" when no perturbation or UQ check was performed.

A useful correction is to phrase results in execution-path terms: "native multilayer estimate" versus "projection-based baseline."

4.6 What py3plex Optimizes For

- reproducible scripting workflows,
- incremental exploration from small to medium-scale multilayer graphs,
- composable query pipelines (especially via DSL in Part III).

4.7 What py3plex Does Not Optimize For

- maximal performance on very large graphs without approximation,
- complete replacement of specialized HPC graph libraries,
- automatic protection from poor modeling assumptions.

4.8 Implications for Users

If your analysis is sensitive to representation choices or approximation error:

1. record provenance and seeds,

2. run at least one robustness check,
3. compare multilayer and flattened baselines explicitly,
4. report implementation path (native multilayer vs delegated/projection path).

Trade-off story: in a commuter network, a projected single-layer betweenness run may finish in seconds and support fast scenario triage, while a native multilayer alternative may take longer but preserve transfer semantics needed for policy claims. The right choice depends on whether the question is screening or inference.

4.9 Why This Matters

py3plex optimizes for explicit, auditable multilayer workflows under practical constraints; it does not optimize for eliminating methodological judgment. Remember this contrast: fast convenience paths are valuable for exploration, but only semantically aligned paths can support strong claims.

CHAPTER

FIVE

INSTALLATION AND FIRST VERIFIED RUN

This chapter gives one supported onboarding path. Alternatives and deployment-heavy setups are in Appendix B.

5.1 Golden-Path Setup

```
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\\Scripts\\activate
pip install --upgrade pip
pip install py3plex
```

If you need editable development mode, install from source in a separate environment. Keep research and development environments distinct.

5.2 Minimal Smoke Test

```
from py3plex.core import multinet
from py3plex.dsl import Q

net = multinet.multi_layer_network(directed=False)
net.add_edges([
    ['Alice', 'social', 'Bob', 'social', 1],
    ['Bob', 'social', 'Cara', 'social', 1],
], input_type='list')

result = Q.nodes().compute('degree').execute(net)
print(result.count)
```

Expected output for this snippet is `3`.

Interpretation note: in this exact snippet, `result.count` is a replica-level count and equals the physical-node count because all edges are in one layer. This is the minimum **verified run** because it checks that import, data structure construction, DSL execution, and result materialization all work in your environment; it is not just a toy print statement.

5.3 First-Run Checks That Actually Matter

After the smoke test:

1. verify your Python version and py3plex version,
2. verify deterministic behavior by setting explicit seeds in stochastic workflows,
3. confirm that your intended file formats are parsed as expected.

5.4 Common Early Failures

- **Import succeeds, query fails:** often missing optional dependencies for specific algorithm families (for example, community-detection extras such as Infomap-related workflows).
- **Unexpected node counts:** replica vs physical-node confusion.
- **Platform mismatch in scripts:** shell-activation differences or path assumptions.

Do not debug these by adding ad-hoc hacks to notebooks; fix environment specification first.

5.5 Where Extended Setup Lives

For Docker, CI images, and deployment-oriented setup, use Appendix B. Keeping those details out of the core narrative prevents onboarding chapters from becoming support manuals.

CHAPTER

SIX

DATA LOADING AND REPRESENTATION CHOICES

Loading data is not a clerical step. Representation decisions made here determine what your metrics can mean later.

6.1 Representation Decisions with Analytical Consequences

Before importing anything, answer:

1. What constitutes a layer in this domain?
2. Are inter-layer edges observed data, inferred links, or modeling conveniences?
3. Are edge weights comparable across layers?
4. What entity identity rules resolve duplicates and missingness?

These decisions should be documented alongside code.

6.2 A Minimal, Auditable Loading Pattern

```
from py3plex.core import multinet

net = multinet.multi_layer_network(directed=False)
net.add_nodes([
    {'source': 'Alice', 'type': 'social'},
    {'source': 'Alice', 'type': 'work'},
    {'source': 'Bob',   'type': 'social'},
])
net.add_edges([
    {'source': 'Alice', 'target': 'Bob',
     'source_type': 'social', 'target_type': 'social'},
])

layers = net.get_layers()
print(layers)
```

Implementation detail: py3plex expects explicit layer attributes in node and edge dictionaries for multilayer semantics.

6.3 Bad Import / Corrected Import (Layer Naming)

```
# Bad: inconsistent layer labels create artificial layer splits
bad_edges = [
    {'source': 'Alice', 'target': 'Bob', 'source_type': 'Social', 'target_type': 'social'},
```

```
]

# Corrected: normalize labels before import
fixed_edges = [
    {'source': 'Alice', 'target': 'Bob', 'source_type': 'social', 'target_type': 'social'},
]
```

6.4 Format Selection (Practical Rule)

Use format choice as a workflow decision:

- **CSV/edgelist:** transparent and easy to diff; good for iterative cleaning.
- **GraphML/GML:** richer metadata, stronger interoperability.
- **Arrow/Parquet pipelines:** useful when throughput and schema stability matter.

No format is universally best. Prefer the one that minimizes ambiguity in your current pipeline.

6.5 Validation Before Analysis

At minimum, validate:

- layer names and counts,
- missing node or layer labels,
- self-loops and duplicate edges (if relevant to your methods),
- weight domain assumptions (non-negative, normalized, etc.).
- explicit distinction between coupling edges and domain edges.

A naive mistake is to accept parser success as data validity.

6.6 Coupling Edges vs Domain Edges

Coupling edges encode identity continuity across layers (for example, `Alice_social` to `Alice_work`), while domain edges encode real relations inside a domain layer (for example, trust or transaction ties). Mixing both without tags makes transfer intensity look like domain connectivity and can distort both centrality and community interpretation.

6.7 What Can Go Wrong

- Layer labels encoded inconsistently (e.g., *Social*, *social*, *soc*).
- Coupling edges mixed with domain edges without tagging.
- Flattened imports accidentally treated as multilayer outputs.
- Cross-layer weights treated as comparable when one layer uses probabilities and another uses counts (for example, 0.8 reliability vs 80 interactions).
- Missingness concentrated in one layer, causing false "low influence" or unstable community assignments for nodes that are merely under-observed.

These errors usually survive until interpretation, where they are expensive to detect.

6.8 Recommendation

Treat data loading code as part of your methodological appendix. If another analyst cannot reconstruct your representation choices from that code, the pipeline is not yet ready. At the end of loading, save a reproducibility bundle containing the cleaned input snapshot, layer schema, entity-resolution rules, and a machine-readable load manifest (versions, parsing options, and checksums).

CHAPTER

SEVEN

VISUALIZATION AS DIAGNOSTIC ANALYSIS

Visualization in multilayer work is best used as a diagnostic instrument, not as proof by picture.

7.1 Analytical Purpose of Plots

Use visualization to test hypotheses about structure:

- Are layers genuinely distinct or mostly redundant?
- Are bridging nodes meaningful connectors or layout artifacts?
- Are inter-layer links interpretable under the domain model?

If the plot cannot answer one of these questions, it is likely decorative.

7.2 A Focused Workflow

```
from py3plex.visualization.multilayer import draw_multilayer_default

layers, layer_graphs, positions = network.get_layers()
# layers: layer-name list; layer_graphs: per-layer graph objects; positions: layout coordinates
draw_multilayer_default((layers, layer_graphs, positions), display=False)
```

Implementation note: layout defaults are convenience choices. They are not statistical estimators.

7.3 Paired-Figure Diagnostic Concept

Use two panels of the same network with different layout seeds. If a bridge narrative appears only in one seed, treat it as a layout-sensitive hypothesis, not a structural fact.

7.4 Common Analyst Failure Modes

1. **"Large node means key actor, ship it."** Often false when size encodes aggregate degree across incomparable layers.
2. **"Dense layer means stronger subsystem."** May reflect data collection bias, sampling intensity, or thresholding rules.
3. **"Clean visual split means true modularity."** May be a force-layout artifact with no robust metric support.

7.5 How to Make Plots More Trustworthy

- annotate what is encoded by size, color, and edge opacity,
- compare at least two layout seeds for stability,
- pair visual claims with numeric checks (coverage, centrality summaries, modularity diagnostics),
- keep layer-specific and global plots side-by-side.

7.6 Plot-to-Metric Follow-Up Workflow

If a plot suggests "Node X is a cross-layer broker," follow immediately with a numeric check (for example, per-layer betweenness plus coverage threshold) before making any interpretive claim.

7.7 When Not to Rely on Visualization

For very large graphs, visual clutter makes node-level interpretation unreliable. Prefer summary statistics and targeted subgraph diagnostics. Multilayer plots become actively misleading when edge density and overlap force occlusion that can fabricate apparent hubs or apparent separations not supported by underlying metrics.

7.8 Conclusion

In this book, visualizations are exploratory and communicative aids. Claims should be grounded in reproducible computations, with plots used to expose potential structure and potential misunderstanding.

CHAPTER

EIGHT

CORE ALGORITHMS: WHAT IS ESTIMATED, WHAT IS APPROXIMATED

This chapter organizes py3plex algorithm use around interpretation risk rather than interface breadth.

8.1 Native Multilayer Methods

These methods preserve layer semantics during estimation (for example, multilayer community workflows, layer-aware coverage logic, and multilayer-specific centrality variants).

- **Safe claim:** "This estimate is computed under an explicit multilayer representation."
- **Unsafe claim:** "Therefore it is automatically closer to domain truth."

8.2 Delegated Single-Layer Methods

Some workflows project or flatten to use mature single-layer backends.

- **Safe claim:** "This is a projection-based baseline for comparison."
- **Unsafe claim:** "This delegated result has the same semantics as native multilayer output."

8.3 Approximate Methods

Approximation is a practical necessity for expensive measures on larger graphs.

- **Safe claim:** "Approximate ranking is acceptable for screening under stated error tolerance."
- **Unsafe claim:** "Small score gaps are interpretable as substantive differences without robustness checks."

8.4 Stochastic Methods

Stochasticity appears in community search, perturbation workflows, and simulation.

- **Safe claim:** "Result is one seeded draw (or one uncertainty summary) under a specified model."
- **Unsafe claim:** "Single seeded output is definitive without stability analysis."

8.5 Community Detection

py3plex supports multilayer community workflows through methods such as Louvain/Leiden-style procedures and related wrappers.

Interpretive caution:

- Partition quality metrics are objective-specific.
- Comparable scores do not imply identical community semantics across methods.
- **Global partitions** optimize communities that can span layers; **per-layer partitions** optimize separate structures inside each layer and are better for layer-to-layer comparison.

8.6 Centrality

Many centrality measures are available, but users must state:

- whether analysis is per-layer or global,
- whether values are exact or approximated,
- whether measures were computed on native multilayer structure or reduced projections.

Approximation is often suitable for ranking-oriented exploration on larger graphs, but claims about small score differences should be avoided unless validated; score gaps that look large in a plot can collapse under perturbation or alternative layer scopes.

Robustness-oriented centrality helpers live alongside the traditional multilayer centrality implementations. Use them when the scientific claim is about rank stability or sensitivity, not only about a single centrality value.

8.7 Embeddings and Representation Learning

The current repository includes embedding APIs in `py3plex.embeddings` and `py3plex.ml.embedding`. These cover NetMF, MetaPath2Vec, Node2Vec, DeepWalk/LINE-style primitives, multiplex variants, link-feature operators, and shared training/evaluation utilities.

Embedding outputs are useful for exploratory similarity, link prediction, and downstream machine-learning features. They should not be described as interpretable multilayer explanations unless the chosen walk schema, meta-paths, negative sampling, and evaluation target are reported.

8.8 Algebraic Paths and Closure

`py3plex.algebra` and `py3plex.semiring` provide semiring-style path, closure, fixed-point, and witness machinery. This is the appropriate family when a workflow needs explicit path algebra (for example shortest paths, reachability, reliability, or lexicographic transfer costs) rather than a single hard-coded shortest-path routine.

Report the semiring, lift function, hop bounds, and witness policy whenever path outputs support a claim.

8.9 Statistical Pooling and Experiment Records

`py3plex.meta` supports fixed-effect and random-effects meta-analysis of network statistics across datasets or experimental conditions. `py3plex` also includes `experiments` helpers for storing run metadata and artifacts.

Use these when a conclusion depends on repeated networks or repeated runs. Do not pool scores across networks without checking whether the estimand and standard errors are comparable.

8.10 Out-of-Core and Planning Infrastructure

`py3plex.out_of_core` provides streaming query execution for disk-resident edge data, and `py3plex.optimizer`/`py3plex.dsl.program` provide planning, cost, and rewrite infrastructure. These are scalability aids. They do not remove the need to state which query shapes are supported and which operations fall back to in-memory analysis.

8.11 Dynamics

SIS/SIR-like models in py3plex are useful for scenario analysis under explicit assumptions.

Do not infer causal epidemiological truths from toy parameter sweeps. Dynamics outputs are model-conditional, not domain truth. Also distinguish topology-driven scenarios (what structure permits) from domain-calibrated scenarios (what parameters and interventions are realistic); both are useful, but they support different claims.

8.12 Practical Pattern

```
from py3plex.dsl import Q, L

result = (
    Q.nodes()
     .from_layers(L['social'])
     .compute('degree', 'betweenness_centrality')
     .order_by('-degree')
     .limit(20)
     .execute(network)
)
```

What this gives: a ranked summary under a specific representation.

What it does not give: robustness, causal explanation, or cross-representation invariance.

8.13 Method Family Summary Table

8.14 Method Selection Checklist

Before choosing an algorithm, specify:

1. target estimand,
2. acceptable approximation error,
3. computational budget,
4. validation strategy (alternative methods, perturbation, or UQ).

This prevents the common workflow error of selecting methods by convenience rather than inferential fit.

CHAPTER

NINE

DSL FUNDAMENTALS: A QUERY MENTAL MODEL

The DSL is useful when you need clear, replayable analytical pipelines over multilayer data. This chapter teaches the mental model first and syntax second.

9.1 Why a DSL Here?

A query pipeline makes assumptions explicit:

- selection scope (which layers, which entities),
- computed measures,
- ordering and filtering logic,
- reproducibility settings.

This helps analysts review and reproduce analytical intent.

9.2 Core Workflow Pattern

```
from py3plex.dsl import Q, L

result = (
    Q.nodes()
     .from_layers(L['social'])
     .where(degree__gt=3)
     .compute('betweenness_centrality')
     .order_by('-betweenness_centrality')
     .limit(10)
     .execute(network)
)
```

Interpretation boundary: this is a reproducible computational selection, not automatically a defensible scientific claim.

9.3 Raw Python vs DSL (When to Use Each)

Use raw Python when the task is truly simple or highly custom (for example, quick one-off filtering in a notebook). Use the DSL when you need an auditable chain with explicit scope, ordering, and reproducibility hooks. A practical split: prototype in Python if needed, then promote claim-bearing analysis to DSL.

9.4 Theory vs Implementation vs Workflow

- **Theory:** filtering by degree defines a set under a degree function.
- **Implementation:** py3plex resolves fields, may autocompute metrics, and executes against internal graph structures.
- **Workflow:** your chosen thresholds and ordering reflect domain judgment.

In the query above, `where(degree__gt=3)` is not just syntax: it operationalizes an inclusion threshold that can change the inferred "important actors" set. Keep these separate in analysis reports.

9.5 Layer Algebra and Scope Control

Layer selection should reflect hypothesis scope, not convenience:

```
from py3plex.dsl import L

social_or_work = L['social'] + L['work']
social_not_work = L['social'] - L['work']
```

A common error is using all layers by default and inferring layer-specific conclusions.

9.6 Autocompute and Hidden Assumptions

Autocompute can be useful, but it can also hide expensive or semantically consequential metric resolution. If a field is resolved implicitly, reviewers may miss that a methodological choice was made. Prefer explicit `.compute(...)` calls in claim-bearing workflows.

9.7 What New Users Usually Misunderstand

1. Query fluency does not remove semantic ambiguity (replicas vs physical nodes).
2. A short query can still encode strong assumptions.
3. Ranking stability requires separate checks (UQ, sensitivity, or perturbation).

For example, `Q.nodes().from_layers(L['social']).order_by('-degree').limit(10)` looks compact but encodes a strong methodological choice: social-layer degree is being treated as the primary definition of importance.

9.8 Next Step

Chapter 10 covers builder internals and explain plans. Chapter 11 covers advanced workflows where query complexity can hide methodological risk.

CHAPTER

TEN

BUILDER API AND EXPLAIN PLANS: READING YOUR OWN QUERY

This chapter focuses on query structure, execution traceability, and failure diagnosis.

10.1 Builder as Structured Intent

The builder API is most valuable when it makes intent reviewable by someone other than the original author.

A useful pattern is to keep pipelines in named stages:

```
from py3plex.dsl import Q, L

scoped = Q.nodes().from_layers(L['social'])
filtered = scoped.where(degree__gt=5)
measured = filtered.compute('degree', 'pagerank')
ranked = measured.order_by('-pagerank').limit(20)
result = ranked.execute(network)
```

This is less concise than one long chain, but easier to audit.

10.2 Explain Plans

Use explain tooling to inspect what will be executed and in what order. This is especially important when autocompute, grouping, or coverage steps are involved.

Ask three questions when reading a plan:

1. Which fields are materialized and when?
2. Which operations depend on grouped context?
3. Where could expensive computations be delayed or reduced?

10.3 Worked Explain-Plan Walkthrough

Consider this query:

```
query = (
    Q.nodes()
     .from_layers(L['social'] + L['work'])
     .where(degree__gt=5)
     .compute('degree', 'betweenness_centrality')
     .order_by('-betweenness_centrality')
```

```
    .limit(15)
)
```

An explain plan should show that layer scoping and filtering happen before expensive centrality computation. Scientifically, that matters because it defines the analysis population before ranking, making the inferential target explicit rather than accidental.

10.4 Opaque Long Chain vs Staged Rewrite

Opaque chain:

```
result = Q.nodes().from_layers(L['*']).where(degree__gt=3).compute('degree', 'pagerank').per_layer().to
```

Improved staged rewrite:

```
scoped = Q.nodes().from_layers(L['*']).where(degree__gt=3)
measured = scoped.compute('degree', 'pagerank')
grouped = measured.per_layer().top_k(10, 'pagerank').end_grouping()
reviewed = grouped.coverage(mode='at_least', k=2).order_by('-pagerank').limit(20)
result = reviewed.execute(network)
```

10.5 Parameterized Queries

For repeated analyses, parameterize thresholds rather than editing literals in notebooks.

```
from py3plex.dsl import Q, Param

query = Q.nodes().where(degree__gt=Param.int('k')).compute('degree')
result = query.execute(network, k=4)
```

This improves reproducibility and reduces accidental drift across runs.

10.6 Failure Diagnostics

When queries fail, categorize the issue quickly:

- syntax-level problem,
- unknown field/measure,
- grouping misuse,
- missing parameters,
- type mismatch.

Treat diagnostic messages as part of your workflow review process, not only as debugging output.

Common failure examples:

```
# Missing parameter
query = Q.nodes().where(degree__gt=Param.int('k')).compute('degree')
# query.execute(network)  # raises parameter-missing error
```

```
# Grouping misuse
bad = Q.nodes().coverage(mode='all')  # coverage without grouping context
```

10.7 Auditable Review: Provenance and AST Hash

Explain plans are not only computational aids. Together with provenance and AST hash, they support auditable scientific review: reviewers can verify the exact query structure that produced a claim and detect silent workflow drift across revisions.

10.8 When the Builder Is Not the Right Tool

If your workflow is a one-off simple filter, direct Python operations may be clearer. DSL helps most when you need replayability, composability, and explicit provenance.

CHAPTER

ELEVEN

ADVANCED WORKFLOWS: COMPOSITIONAL QUERIES UNDER UNCERTAINTY

Advanced DSL usage is not about longer chains. It is about controlling semantic scope, computational cost, and uncertainty propagation.

11.1 Workflow Pattern 1: Grouped Selection with Coverage

```
from py3plex.dsl import Q, L

result = (
    Q.nodes()
     .from_layers(L['*'])
     .compute('degree')
     .per_layer()
       .top_k(10, 'degree')
     .end_grouping()
     .coverage(mode='at_least', k=2)
     .execute(network)
)
```

Judgment point: coverage threshold controls strictness. `all` can be too restrictive in heterogeneous systems. Do not use coverage when the analysis target is deliberately layer-specific; cross-layer intersection logic can erase exactly the heterogeneity you are trying to study.

11.2 Workflow Pattern 2: UQ-Aware Ranking

```
result = (
    Q.nodes()
     .compute('pagerank')
     .uq(method='bootstrap', n_samples=100, seed=42)
     .order_by('-pagerank')
     .limit(20)
     .execute(network)
)
```

Interpretive warning: confidence intervals quantify algorithm/data perturbation under a model; they do not validate domain truth. Interpretive warning: interval overlap does not imply irrelevance, and non-overlap does not imply biological or social truth; both depend on model choice, data quality, and estimand definition.

11.3 Workflow Pattern 3: Temporal Slicing

```
temporal = (
    Q.edges()
     .during(100.0, 150.0)
     .from_layers(L['transport'])
     .execute(temporal_network)
)
```

Judgment point: time-window boundaries can dominate observed effects. Report window design choices explicitly. Here `.during()` is intended as stable public DSL syntax for temporal windowing, not merely illustrative pseudocode.

11.4 Composability vs Readability

As pipelines grow, readability declines. Recommended practices:

- split complex pipelines into named stages,
- log final query strings or AST hashes,
- avoid mixing exploratory and production logic in one chain.

11.5 Performance and Approximation

For larger networks:

- use selective layer scopes before expensive measures,
- use approximations when ranking is sufficient,
- record approximation parameters in outputs.

Never present approximate outputs as exact without explicit disclosure.

11.6 Static Review Before Execution

Current DSL tooling includes a static-analysis layer in `py3plex.dsl.lint`. Use it before long or expensive runs when a query depends on fragile layer names, schema assumptions, or expensive full scans.

```
from py3plex.dsl import Q
from py3plex.dsl.lint import lint

query = Q.nodes().where(degree__gt=5).compute('pagerank').to_ast()
diagnostics = lint(query, graph=network)
for diagnostic in diagnostics:
    print(diagnostic.code, diagnostic.message)
```

This does not replace execution-time validation. It is an early warning layer for unknown layers, unknown attributes, type mismatches, redundant predicates, and performance hazards.

11.7 Graph Programs and Rewrites

For reusable workflows that must be inspected, transformed, or compared, `py3plex.dsl.program` exposes `GraphProgram` objects. The practical use case is not "more abstraction"; it is auditable program identity, rewrite history, cost estimates, and execution fingerprints.

```
from py3plex.dsl import Q
from py3plex.dsl.program import GraphProgram, apply_rewrites

program = GraphProgram.from_ast(Q.nodes().compute('degree').to_ast())
optimized = apply_rewrites(program)
result = optimized.execute(network)
```

When reviewing a rewritten program, verify that optimization did not change the analysis population, grouping semantics, or uncertainty configuration.

11.8 Scaling and Cross-Run Workflows

The advanced DSL ecosystem now includes adjacent infrastructure:

- `py3plex.out_of_core` for supported streaming query shapes over disk-backed edge data,
- `py3plex.meta` for pooling comparable network statistics across networks,
- `py3plex.experiments` for persisting run metadata and artifacts.

These tools are most valuable when they preserve the same query semantics while making larger or repeated analyses auditable. They should not be used to hide unsupported query patterns or incompatible effect definitions.

11.9 Advanced Checklist

Before finalizing an advanced query workflow:

1. verify semantic scope (global vs per-layer),
2. verify uncertainty configuration,
3. verify seeds and provenance capture,
4. verify that outputs answer the intended question.

11.10 Synthesis: Managing Complexity Creep

Advanced pipelines fail most often through complexity creep: each added clause is locally reasonable, but the combined workflow becomes hard to audit. Treat readability, explicit assumptions, and provenance capture as first-class constraints, not post-hoc cleanup tasks.

CHAPTER

TWELVE

LIMITATIONS AND STABILITY: WHAT PY3PLEX DOES NOT PROMISE

This chapter is intentionally strict. It defines reliability boundaries for methods used in this book.

12.1 Scope of Stability Statements

A statement like "stable API" means the workflows documented here are intended to remain usable across minor revisions. It does **not** mean every internal behavior is immutable.

12.2 Major Limitation Classes

1. **Semantic limitations** Multilayer outputs can be misread if replica-level and physical-node interpretations are mixed.
2. **Algorithmic limitations** Some methods rely on assumptions (connectivity, weight domains, stochastic convergence) that may fail silently if not checked.
3. **Scalability limitations** Exact methods can become impractical on larger graphs; approximations trade precision for tractability.
4. **Interoperability limitations** Delegation to single-layer backends can alter semantics if projection steps are not explicit.

12.3 Stability Practices That Help

- pin versions for reproducible studies,
- record query provenance and random seeds,
- include at least one robustness comparison,
- keep method caveats near reported results.

12.4 What to Report in Publications

At minimum, report:

- representation choices,
- algorithm and parameter settings,
- approximation/UQ configuration,
- software version and environment details,
- known limitations relevant to your claims.

A short, honest limitations paragraph is usually more credible than broad claims of robustness.

12.5 What We Would Need to Trust Stronger Claims

Stronger claims would require converging evidence across representation alternatives, robustness checks that preserve the substantive conclusion, and validation against at least one external signal (for example, known events, curated benchmarks, or domain-grounded labels).

12.6 Example Limitations Paragraph for a Paper

"Our multilayer ranking is conditional on the layer schema, coupling assumptions, and perturbation model used in this study. Projection-based baselines and per-layer alternatives were evaluated and produced qualitatively similar high-level patterns, but several node-level rank swaps remained sensitive to representation and temporal window choice. Therefore we interpret top-tier findings as robust trends and lower-tier differences as model-conditional."

This chapter should be read alongside Chapter 3 (semantic ambiguity at replica vs physical level) and Chapter 8 (algorithm-family-specific safe and unsafe claims).

CHAPTER

THIRTEEN

CASE STUDY 1 — SOCIAL MULTIPLEX: INFLUENCE VS BROKERAGE

13.1 Research Question

How does influence ranking change when friendship, collaboration, and mentorship ties are modeled as separate layers instead of a single flattened graph?

13.2 Data and Representation Choices

We represent each person as replicas across three layers:

- `friendship`
- `collaboration`
- `mentorship`

Contestable choice: inter-layer edges connect replicas of the same person with uniform coupling weight. This simplifies identity continuity but may underrepresent context switching costs.

13.3 Baseline: Flattened Analysis

```
flat = network.flatten_to_monoplex(method='union')
# Run standard centrality on flattened graph
```

Flattened ranking highlights well-connected generalists. It cannot distinguish whether influence comes from one dominant layer or cross-layer brokerage.

13.4 Multilayer Analysis

```
from py3plex.dsl import Q, L

per_layer = (
    Q.nodes()
     .from_layers(L['friendship'] + L['collaboration'] + L['mentorship'])
     .compute('degree', 'betweenness_centrality')
     .per_layer()
       .top_k(15, 'betweenness_centrality')
     .end_grouping()
     .coverage(mode='at_least', k=2)
```

```
    .execute(network)
)
```

Key finding: several individuals absent from flattened top-k appear repeatedly as high-brokerage nodes across layers.

13.5 Tiny Result Table (Illustrative)

Interpretive shift: P12 looks secondary in flattened degree but becomes first-ranked under cross-layer brokerage because it links mentorship and collaboration substructures.

13.6 Why Multilayer Changed the Result

Flattening merged semantically different ties and amplified dense collaboration clusters. The multilayer query isolated cross-context connectors that are structurally modest in any single layer but strategically important across layers.

13.7 Fragile Assumptions

1. Uniform inter-layer coupling may overstate identity continuity.
2. Missingness differs by layer (mentorship ties are often underreported).
3. Top-k thresholds are sensitive to layer size imbalance.

13.8 Robustness Checks

```
uq_result = (
    Q.nodes()
     .compute('betweenness_centrality')
     .uq(method='bootstrap', n_samples=100, seed=42)
     .execute(network)
)
```

We treat rank changes within confidence overlap as ambiguous rather than definitive. Under bootstrap/UQ, the top-3 set remained stable while positions 4–8 swapped frequently, so we report a robust core and a contingent middle tier rather than a single rigid ordering.

13.9 Transferable Lesson

If the practical question concerns cross-context brokerage, flattened centrality is usually a poor proxy. Layer-aware selection with explicit coverage criteria is more informative.

13.10 Local Caveat

This case relies on relatively stable layer definitions. If labels such as “mentorship” and “collaboration” are noisy or drift over time, cross-layer brokerage can be spuriously inflated or suppressed, directly altering who appears “strategically central.”

CHAPTER

FOURTEEN

CASE STUDY 2 — BIOLOGICAL MULTILAYER: ROBUST SIGNAL OR LAYER ARTIFACT?

This chapter is written as an inference audit for biological interpretation: the central question is not only "which genes rank high," but whether that ranking survives mechanistic separation of layers.

14.1 Research Question

Do high-centrality genes remain high across interaction layers (PPI, regulation, co-expression), or are "hub" claims layer artifacts?

14.2 Representation and Contestable Choices

Layers:

- `ppi` (physical interactions)
- `regulatory`
- `coexpression`

Contestable choice: we treat all edges as unweighted in the baseline, then compare against a weighted sensitivity run to test whether confidence scores materially reorder the candidates.

14.3 Naive Alternative

A flattened analysis can identify global hubs quickly, but it mixes mechanistically different relations and tends to overweight denser layers.

14.4 Layer-Aware Workflow

```
from py3plex.dsl import Q, L

result = (
    Q.nodes()
     .from_layers(L['ppi'] + L['regulatory'] + L['coexpression'])
     .compute('degree', 'pagerank', 'betweenness_centrality')
     .per_layer()
       .top_k(25, 'pagerank')
     .end_grouping()
     .coverage(mode='at_least', k=2)
```

```
    .execute(network)
)
```

Interpretation: nodes retained after coverage filtering are less likely to be single-layer artifacts.

14.5 Uncertainty and Stability

```
stable = (
    Q.nodes()
     .compute('pagerank')
     .uq(method='stratified_perturbation', n_samples=80, seed=42)
     .execute(network)
)
```

This estimates ranking stability under structured perturbation. We do not interpret narrow intervals as biological certainty; they quantify model-internal stability only. In our weighted-vs-unweighted sensitivity comparison, 4 of the top 10 genes remained unchanged, 3 moved within the top 10, and 3 dropped out when confidence weights were applied.

14.6 What Flattening Missed

Flattening promoted genes central in co-expression only. This is not just a technical nuisance: co-expression density can reflect assay-wide covariance that is biologically broad but mechanistically weak, so flattening can overprioritize correlation-rich modules over regulatory or interaction-critical genes. Multilayer filtering identified a smaller set with repeated prominence across mechanistically different layers, yielding candidates more consistent with cross-pathway involvement.

14.7 Fragile Assumptions

1. Layer construction pipelines can introduce correlated bias.
2. Edge confidence scores are heterogeneous across sources.
3. Inter-layer coupling choice affects cross-layer persistence claims.

Operational note: in this chapter, "cross-layer persistence" means a gene remains in the top-k under at least two mechanistically distinct layers after per-layer ranking and coverage filtering.

14.8 Reproducibility Practices

- fixed random seeds,
- archived input snapshots,
- recorded query configuration,
- reported software versions and environment metadata.

14.9 Transferable Lesson

In biological multilayer studies, "important gene" claims should usually be phrased as model-conditional and layer-conditional unless cross-layer persistence is demonstrated; this is specifically a guard against mistaking co-expression abundance for multi-mechanism relevance.

CHAPTER

FIFTEEN

CASE STUDY 3 — TRANSPORTATION NETWORK: RESILIENCE UNDER LAYER DISRUPTION

This chapter focuses on transportation resilience claims that can be defended under explicit disruption scenarios.

15.1 Research Question

Which stations are critical for multimodal resilience, and how does that answer change when one mode is degraded?

15.2 Representation Choices

Layers:

- `metro`
- `bus`
- `bike`
- `walk_transfer`

Simplifying assumption: transfer edges are modeled with fixed penalty weights, though in practice transfer costs vary by time, congestion, and accessibility.

15.3 Naive Baseline

A flattened shortest-path analysis finds globally short routes but hides mode dependency. It cannot separate "fast because metro exists" from "resilient across mode failures."

15.4 Multilayer Resilience Workflow

```
from py3plex.dsl import Q, L

critical = (
    Q.nodes()
     .from_layers(L['metro'] + L['bus'] + L['bike'])
     .compute('betweenness_centrality')
     .per_layer()
       .top_k(20, 'betweenness_centrality')
     .end_grouping()
     .coverage(mode='at_least', k=2)
```

```
    .execute(network)
)
```

Then run a disruption scenario by removing or down-weighting one layer and recomputing rankings. Example scenario: reduce metro edge capacity by 40% to emulate a line-level outage during peak service and recompute top brokerage nodes.

15.5 Why Multilayer Changed the Conclusion

Flattened analysis prioritized metro-core stations because it did not account for mode-specific dependency. Under the metro-degradation scenario, Station S04 moved from rank 11 to rank 2 while Station S01 dropped from rank 1 to rank 6, revealing contingency that flattened ranking hid.

15.6 Fragile Assumptions

1. Temporal aggregation can erase peak-hour fragility (for example, averaging 07:00–09:00 with midday can hide commuting bottlenecks).
2. Static transfer penalties may understate accessibility constraints.
3. Missing pedestrian connectivity biases resilience estimates.

15.7 Reproducibility and Auditability

- fixed seeds for any stochastic components,
- explicit scenario definitions (what is removed or perturbed),
- stored query/provenance metadata for each scenario,
- deterministic export of summary tables.

15.8 Transferable Lesson

For transport planning questions about disruption, multilayer scenario analysis is often more informative than flattened efficiency metrics, but resilience must be stated precisely: resilience of which service objective (for example, retained accessibility), under which degradation model, measured by which metric.

15.9 Local Caveat

This workflow is strongest for topology-driven resilience screening. It is not a substitute for full demand, schedule, or behavioral models; those models can separate interchange importance (transfer criticality) from mode dependence (vulnerability to one transport mode failing).

CHAPTER

SIXTEEN

TESTING AND VALIDATION AS METHODOLOGICAL CONTROLS

Testing is not only software hygiene. In analytical pipelines, tests are controls against silent methodological drift (such as changes in representation or approximation defaults that alter results).

16.1 Validation Layers

py3plex workflows benefit from four test layers:

1. **Unit checks** for deterministic core behavior.
2. **Property/metamorphic checks** for invariants under transformations.
3. **Differential checks** across equivalent APIs.
4. **Workflow checks** that validate representative end-to-end analyses.

16.2 Mini-Test Example (Main-Text Scale)

```
from py3plex.dsl import Q

result = Q.nodes().compute('degree').execute(network)
assert result.count > 0
assert 'degree' in result.to_pandas().columns
```

This tiny test does not prove correctness, but it catches a surprisingly common regression class: query executes but expected metric materialization changes silently.

16.3 Why This Matters for Analysis

A pipeline can execute successfully and still be wrong:

- representation drift changes semantics,
- approximation defaults change outputs (for example, an exact betweenness call silently becoming approximate),
- query refactors alter grouping logic.

Tests should target these failure modes directly.

16.4 Practical Workflow

- run focused tests near changed analytical components,
- include at least one deterministic seed-based test for stochastic paths,
- preserve small synthetic fixtures with known expected behavior,
- treat regression diffs as analytical review prompts, not only coding errors.

16.5 Current Repository Validation Map

The repository now contains focused tests for the newer subsystems discussed in this edition:

- `tests/test_dsl_lint.py` for static DSL diagnostics,
- `tests/test_program.py` and `tests/test_dsl_program_rewrite.py` for `GraphProgram` identity, typing, and rewrite behavior,
- `tests/test_algebra_*.py` and `tests/property/test_algebra_properties.py` for semiring/algebra invariants,
- `tests/test_meta_analysis.py` and `tests/property/test_meta_properties.py` for pooled-effect semantics,
- `tests/test_out_of_core.py` for streaming-query boundaries,
- `tests/test_metapath2vec.py` and embedding examples for representation learning behavior.

Use these files as starting points when changing the corresponding package families. They are more reliable than searching for a broad "full suite" entry point because each subsystem has different optional dependencies and runtime costs.

16.6 Example Validation Questions

- Does node/edge count conservation hold after import transformations?
- Do equivalent query formulations return equivalent sets (for example, `Q.nodes().where(layer='social')` versus a graph-ops layer filter path)?
- Are ranking changes under perturbation within expected bounds?
- Are provenance fields complete and stable?

When a test fails, treat it as an analytical review decision point: decide whether the underlying methodological change is intended and document it, or roll it back.

16.7 Connection to Reproducibility

Testing catches accidental changes; reproducibility practices make intentional changes auditable. Both are required for credible technical results. The next chapter turns this into environment-level practice: how to make a run replayable across machines and time.

CHAPTER

SEVENTEEN

REPRODUCIBLE ENVIRONMENTS AND REPLAYABLE ANALYSIS

Reproducibility is an analytical requirement: independent reviewers must be able to re-run your workflow and verify the assumptions made and parameters used in the analysis.

For basic environment setup commands, see Chapter 5. For container/deployment detail, see Appendix B.

17.1 Reproducibility Stack

A reproducible py3plex study should preserve:

1. dependency versions,
2. data snapshots or immutable references,
3. query definitions and parameters,
4. random seeds,
5. execution metadata/provenance.

17.2 Minimal Practice Pattern

- pin package versions in environment files,
- record py3plex version and Python version in outputs,
- store query text/AST and parameter bindings,
- store seeds for stochastic methods,
- save result artifacts with timestamps and scenario labels.

17.3 What Reproducibility Does Not Guarantee

A replayable pipeline can still encode weak assumptions. Reproducibility supports auditability; it does not prove substantive validity.

17.4 Common Failure Modes

- hidden notebook state,
- unpinned dependencies,
- overwritten intermediate files,
- undocumented manual edits,

- inconsistent data extraction windows.

17.5 Mitigation

- use scriptable pipelines over ad-hoc notebook-only workflows,
- keep immutable raw inputs separate from derived data,
- include an explicit run manifest with versions, seeds, and hashes.

17.6 Concrete Run-Manifest Example

```
run_id: social_multiplex_2026_03_22_001
py3plex_version: 2.0.92
python_version: 3.12.3
git_commit: f2ffa0813a9c
dataset_checksum: sha256:2a0c...9f4b
query_ast_hash: 4f8a73c2b1d8e9aa
seed: 42
output_bundle: results/social_multiplex_2026_03_22_001.bundle.json.gz
```

From DSL execution, preserve at minimum: query AST hash, bound parameters, layer scope, seed/randomness configuration, and network fingerprint fields (node/edge/layer counts).

17.7 Link to Scientific Credibility

In multilayer analysis, representation and parameter choices are often contestable. Reproducibility makes those choices inspectable and debatable rather than opaque. Counterexample: two teams can reproduce the same flattened-ranking script bit-for-bit and still reach a substantively invalid conclusion if the chosen representation erased the layer semantics needed for the scientific question.

CHAPTER

EIGHTEEN

GUI OVERVIEW: WHERE IT HELPS, WHERE IT DOES NOT

The py3plex GUI is a convenience interface for interactive exploration, useful for rapid inspection and teaching workflows. It should not replace versioned analytical scripts.

Status: Experimental

The GUI is intended for local or controlled environments. It is not a hardened public deployment target.

18.1 Appropriate Uses

- quick dataset inspection,
- exploratory layer browsing,
- demonstration and teaching,
- rapid hypothesis sketching before scripted analysis.

18.2 One Short Interaction Flow

1. Load a multilayer dataset and inspect layer coverage in the GUI.
2. Run a quick centrality ranking to spot candidate bridge nodes.
3. Export the selected query/configuration and rerun the claim-bearing analysis in a versioned script.

18.3 Inappropriate Uses

- sole source of publication-critical results,
- unversioned "click-through" analysis without provenance,
- security-sensitive internet-facing deployment.

18.4 GUI-to-Script Handoff Example

If the GUI suggests a candidate query such as "top 20 nodes by degree in social and work layers," reproduce it in code (for example, `Q.nodes().from_layers(L['social'] + L['work']).compute('degree').order_by('-degree').limit(20)`) and commit that script with explicit seeds and exports.

18.5 What "Experimental" Means Operationally

Here, "experimental" means all three: unstable interface expectations over releases, incomplete provenance capture relative to scripted workflows, and higher deployment risk outside controlled environments.

18.6 Recommended Practice

Use the GUI to discover questions, then transfer finalized analysis into scriptable py3plex workflows with explicit parameters, seeds, and exports.

This keeps interactive exploration and reproducible inference aligned rather than conflated.

18.7 Recommendation Matrix

- **Use GUI for:** exploratory inspection, teaching demos, quick visual hypothesis generation.
- **Do not use GUI for:** final publication-grade inference, untracked parameter exploration, or public deployment without additional hardening and audit controls.

CHAPTER

NINETEEN

APPENDIX A: REPOSITORY LAYOUT AND SCRIPTS

This appendix is a compact navigation map, not a full manual.

19.1 Repository Map (Condensed)

```
py3plex/
├── py3plex/      # Main package (core, algorithms, dsl, algebra, embeddings, io, ...)
├── tests/        # Automated validation and regression checks
├── examples/     # Runnable examples by topic
├── book/         # This manuscript source
├── docfiles/     # Sphinx documentation source
├── gui/          # Experimental web interface
├── pyproject.toml
└── Makefile
```

The main package is organized around analytical concerns: representation (`core/`), inference (`algorithms/` and `centrality/`), query workflows (`dsl/`), simulation (`dynamics/`), data interchange (`io/` and `out_of_core/`), algebraic path/query machinery (`algebra/` and `semiring/`), embeddings (`embeddings/` and `ml/embedding/`), experiment tracking (`experiments/`), uncertainty quantification (`uncertainty/`), visual outputs (`visualization/`), meta-analysis (`meta/`), and cost/planning support (`optimizer/` and `dsl/program/`).

19.2 Current Package Landmarks

Use this list to orient yourself when the book discusses newer repository capabilities:

`py3plex/algebra` and `py3plex/semiring`
: Semiring-style path, closure, fixed-point, and witness utilities.

`py3plex/embeddings` and `py3plex/ml/embedding`
: NetMF, MetaPath2Vec, Node2Vec/DeepWalk/LINE-style embedding primitives and shared embedding utilities.

`py3plex/dsl/lint` and `py3plex/dsl/program`
: Static query diagnostics plus first-class `GraphProgram` objects for typed, rewritable, cost-aware DSL workflows.

`py3plex/experiments` and `py3plex/meta`
: Filesystem-backed experiment records and meta-analytic pooling of network statistics across runs or datasets.

`py3plex/out_of_core` and `py3plex/optimizer`
: Streaming query execution over disk-resident edge data and cost-based planning infrastructure for larger analyses.

`py3plex/uncertainty` and `py3plex/visualization`
: Bootstrap, perturbation, and partition uncertainty utilities plus plotting and rendering helpers for multilayer outputs.

19.3 Chapter-to-Example Map

Installation and First Verified Run **(page 11)**
`examples/getting_started/` for minimal verified runs.

Data Loading and Representation Choices **(page 13)**
`examples/io_and_data/` for loading and format-conversion patterns.

Core Algorithms: What Is Estimated, What Is Approximated **(page 18)**
`examples/network_analysis/` for centrality, communities, and dynamics.

Visualization as Diagnostic Analysis **(page 16)**
`examples/visualization/` for rendering-oriented examples.

DSL Fundamentals: A Query Mental Model **(page 21) and** *Advanced Workflows: Compositional Queries Under Uncertainty* **(page 26)**
`examples/dsl_zoo/` for DSL patterns and reusable snippets.

Advanced Workflows: Compositional Queries Under Uncertainty **(page 26)**
`examples/advanced/example_graph_program.py` and `examples/advanced/example_rewrite_engine.py` for program/rewrite examples; `examples/out_of_core/` for streaming query examples.

Core Algorithms: What Is Estimated, What Is Approximated **(page 18)**
`examples/advanced/example_metapath2vec.py` and related embedding examples for the current embedding surface.

Advanced Workflows: Compositional Queries Under Uncertainty **(page 26)**
`examples/pipelines/` for pipeline and workflow composition examples.

GUI Overview: Where It Helps, Where It Does Not **(page 41)**
`gui/` for local GUI setup and interaction flow.

19.4 Where to Start to Reproduce Book Examples

1. Create a clean environment and run the chapter-5 smoke test first.
2. Start with `examples/getting_started/` and then open the chapter-mapped folders above.
3. Use one case-study script at a time, keeping seeds and output paths explicit.
4. Record version, commit, and query/provenance metadata alongside outputs.

19.5 Key Commands (Minimal)

```
make setup
make dev-install
make test
make docs
```

These commands cover most local reproduction workflows; deployment-heavy details are intentionally moved to Appendix B and repository docs.

CHAPTER

TWENTY

APPENDIX B: DOCKER, DOCKER COMPOSE, AND DEPLOYMENT

This appendix keeps one recommended deployment path for controlled environments and highlights caveats that matter for analytical integrity.

20.1 Recommended Path (Controlled Environment)

1. Build one pinned image for the analysis environment.
2. Run py3plex workflows in that container with mounted input/output directories.
3. Keep GUI usage local or inside trusted internal networks only.

```
# Build image
docker build -t py3plex:latest .

# Build with specific Python version
docker build --build-arg PYTHON_VERSION=3.11 -t py3plex:3.11 .

# Build with version tag matching book release
docker build -t py3plex:2.0.92 .

# Run a pinned reproducible analysis
docker run --rm -v $(pwd)/data:/data -v $(pwd)/results:/results py3plex:2.0.92 python script.py
```

For multi-service local orchestration:

```
docker compose up -d
docker compose logs -f
docker compose down
```

20.2 Security and Boundary Warnings

Warning

The GUI is experimental and should not be treated as a hardened public service. Use controlled, trusted environments unless you add independent hardening, authentication, and operational monitoring.

At minimum:

- run with explicit secrets via environment variables,

- prefer non-root containers,
- restrict uploads and writable paths,
- terminate TLS if exposed beyond localhost/VPN,
- keep base images and dependencies updated.

20.3 What This Appendix Deliberately Omits

To keep the book focused, broad cloud deployment matrices and command-heavy infrastructure permutations are maintained in repository documentation rather than the manuscript body.

See also:

- *GUI Overview: Where It Helps, Where It Does Not* (page 41) for GUI usage boundaries,
- repository deployment docs for extended cloud and reverse-proxy recipes.

CHAPTER

TWENTYONE

APPENDIX C: DETAILED VALIDATION SCRIPTS

This appendix complements *Testing and Validation as Methodological Controls* (page 37) by grouping script patterns by methodological principle rather than by tool catalog.

21.1 What to Run First

Prioritize, in order:

1. conservation/invariant checks on small fixtures,
2. differential checks across equivalent APIs,
3. perturbation/UQ stability checks for claim-bearing outputs.

These three catch most analytical drift early.

21.2 Principle 1: Conservation and Structural Invariants

Use compact tests to verify invariants such as probability conservation in random walks, population conservation in SIS/SIR trajectories, and valid bounds for modularity or centrality outputs.

```
# pseudocode: conservation-style check
result = run_model(network, seed=42)
assert invariant_holds(result)
```

21.3 Principle 2: Differential Equivalence

Check that equivalent analysis paths agree when they should.

```
# pseudocode: equivalent query paths
a = dsl_query_path(network)
b = equivalent_graph_ops_path(network)
assert compare_semantics(a, b)
```

21.4 Principle 3: Stability Under Perturbation

For rank- or partition-based claims, include perturbation or bootstrap checks and report what changed.

```
# pseudocode: stability envelope
base = compute_ranking(network)
```

```
perturbed = [compute_ranking(perturb(network, seed=s)) for s in seeds]
summarize_rank_stability(base, perturbed)
```

21.5 Methodological Framing

These scripts are not only software QA artifacts. They are methodological controls that bound overinterpretation by testing whether conclusions survive equivalent formulations, perturbations, and known invariants.

For executable, repository-specific script variants, see the `tests/` and `tests/property/` directories referenced in *Testing and Validation as Methodological Controls* (page 37).

CHAPTER

TWENTYTWO

APPENDIX D: ERROR HANDLING AND EXCEPTION HIERARCHY

This appendix is a practical reference for the exceptions analysts hit most often.

22.1 Hierarchy at a Glance

```
Py3plexException
├── Validation / input errors
├── I/O and serialization errors
├── DSL query errors
├── Algorithm execution/convergence errors
└── Configuration errors
```

22.2 High-Frequency Exceptions in Workflow Practice

- **InvalidLayerError / UnknownLayerError** Usually indicates schema mismatch or layer-label drift at load time.
- **QuerySyntaxError / DslSyntaxError** Indicates malformed DSL expressions or invalid builder chaining.
- **QueryExecutionError / DslExecutionError** Indicates runtime mismatch (missing metric, unsupported operation, grouping misuse).
- **SerializationError / FileFormatError** Indicates import/export mismatch or malformed files.
- **ConvergenceError** Indicates iterative algorithms did not stabilize under provided settings.

22.3 Workflow-Review Use

Exception categories help distinguish software faults from methodological faults. A layer-name error suggests data/schema review; a grouping misuse error suggests query-design review; a convergence error suggests algorithm/parameter sensitivity review.

22.4 Minimal Handling Pattern

```python
from py3plex.exceptions import Py3plexException

try:
    result = run_analysis(network)
except Py3plexException as e:
    # Log context, then route to data/query/algorithm review
    print(f"py3plex workflow error: {e}")
```

For pedagogical DSL error examples and recovery patterns, see Chapters 9–11.

CHAPTER

TWENTYTHREE

APPENDIX E: EXTENDED API AND DSL REFERENCE

This appendix is lookup-oriented reference. Conceptual interpretation belongs in Chapters 9–11 and foundational semantics in Chapters 2–4.

23.1 Core API (Minimal Reference)

```
from py3plex.core import multinet
net = multinet.multi_layer_network(directed=False)
net.add_edges([['A', 'social', 'B', 'social', 1]], input_type='list')
layers = net.get_layers()
```

23.2 DSL Builder Quick Reference

```
from py3plex.dsl import Q, L

result = (
    Q.nodes()
     .from_layers(L['social'])
     .where(degree__gt=3)
     .compute('degree')
     .order_by('-degree')
     .limit(10)
     .execute(net)
)
```

23.3 String DSL Quick Reference

```
from py3plex.dsl import execute_query
execute_query(net, 'SELECT nodes WHERE layer="social" COMPUTE degree')
```

23.4 Common Measures

`degree`, `betweenness_centrality`, `closeness_centrality`, `pagerank`, `clustering`

23.5 Current Advanced Surfaces

This compact reference lists the repository additions used elsewhere in the book. Treat these as entry points, then inspect module docstrings and examples for supported arguments.

py3plex.dsl.lint
: `lint(query_ast, graph=network)` and `explain(query_ast, graph=network)` for static diagnostics before execution.

py3plex.dsl.program
: `GraphProgram.from_ast(...)`, `apply_rewrites(...)`, `estimate_program_cost(...)`, and program fingerprints for reusable DSL workflows.

py3plex.algebra / py3plex.semiring
: Semiring registry, path, closure, fixed-point, and witness utilities.

py3plex.embeddings / py3plex.ml.embedding
: NetMF, MetaPath2Vec, Node2Vec, DeepWalk, LINE, multiplex embeddings, link operators, and evaluation helpers.

py3plex.out_of_core
: `OutOfCoreNetwork` plus streaming execution helpers for supported disk-backed query patterns.

py3plex.meta and py3plex.experiments
: Meta-analysis builders/results and filesystem-backed experiment records for reproducible multi-run comparisons.

23.6 Result Export

```
df = result.to_pandas()
result.to_json()
```

23.7 CLI Hint

```
python -m py3plex --help
```

23.8 Reference Boundary Note

If semantics or claim interpretation is at stake, return to Chapters 2–4 and 9–11. This appendix intentionally omits methodological argument and focuses on navigable syntax lookup.

CHAPTER

TWENTYFOUR

BIBLIOGRAPHY

This section provides references for further reading on multilayer networks and py3plex.

24.1 How to Cite This Book

Use the following citation for this manuscript edition:

```
Škrlj, B. (2025). Practical Multilayer Network Analysis with Py3plex (Version 2.0.92).
https://github.com/SkBlaz/py3plex
```

BibTeX:

```
@book{Skrlj2025book,
  author = {Škrlj, Blaž},
  title = {Practical Multilayer Network Analysis with Py3plex},
  year = {2025},
  version = {2.0.92},
  url = {https://github.com/SkBlaz/py3plex}
}
```

24.2 Core References

24.3 Py3plex Publications

24.4 Community Detection

24.5 Network Embeddings

24.6 Dynamics and Processes

24.7 Applications

24.7.1 Biological Networks

24.7.2 Social Networks

24.7.3 Transportation Networks

24.8 Software and Tools

24.9 Online Resources

Py3plex:

- Documentation: https://skblaz.github.io/py3plex/
- GitHub: https://github.com/SkBlaz/py3plex
- PyPI: https://pypi.org/project/py3plex/

Community:

- Network Science community: https://netscisociety.net/
- Complex Networks: https://www.complexnetworks.org/

Datasets:

- Network Repository: https://networkrepository.com/
- SNAP: Stanford Network Analysis Project: https://snap.stanford.edu/data/
- Netzschleuder: https://networks.skewed.de/

24.10 Recommended Reading Path

24.10.1 For Graduate Students

1. Start with [Kivelä2014] for comprehensive foundations
2. Read [Skrlj2019a] for py3plex overview
3. Read [Mucha2010] for community detection
4. Explore [Bianconi2018] textbook for deeper theory

24.10.2 For Practitioners

1. Start with [Skrlj2019a] for py3plex capabilities
2. Skim [Kivelä2014] for key concepts
3. Focus on application papers relevant to your domain
4. Use this book and online docs for implementation

24.10.3 For Researchers

1. Read [Kivelä2014] and [Boccaletti2014] thoroughly
2. Study [Domenico2013] for mathematical rigor
3. Read domain-specific application papers
4. Explore [Bianconi2018] for advanced topics

BIBLIOGRAPHY

[Kivelä2014] Kivelä, M., Arenas, A., Barthelemy, M., Gleeson, J. P., Moreno, Y., & Porter, M. A. (2014). Multilayer networks. *Journal of Complex Networks*, 2(3), 203-271. https://doi.org/10.1093/comnet/cnu016

Comprehensive review of multilayer network theory, mathematics, and applications. Essential reading.

[Boccaletti2014] Boccaletti, S., Bianconi, G., Criado, R., Del Genio, C. I., Gómez-Gardenes, J., Romance, M., … & Zanin, M. (2014). The structure and dynamics of multilayer networks. *Physics Reports*, 544(1), 1-122. https://doi.org/10.1016/j.physrep.2014.07.001

Physics-focused perspective on multilayer networks with emphasis on dynamics.

[Bianconi2018] Bianconi, G. (2018). *Multilayer Networks: Structure and Function*. Oxford University Press.

Comprehensive textbook covering theory, methods, and applications.

[Domenico2013] De Domenico, M., Solé-Ribalta, A., Cozzo, E., Kivelä, M., Moreno, Y., Porter, M. A., … & Arenas, A. (2013). Mathematical formulation of multilayer networks. *Physical Review X*, 3(4), 041022. https://doi.org/10.1103/PhysRevX.3.041022

Rigorous mathematical formulation of multilayer network representations.

[Skrlj2019a] Škrlj, B., Kralj, J., & Lavrač, N. (2019). Py3plex toolkit for visualization and analysis of multilayer networks. *Applied Network Science*, 4(1), 94. https://doi.org/10.1007/s41109-019-0203-7

Primary py3plex paper. Cite this when using py3plex in research.

[Skrlj2019b] Škrlj, B., Kralj, J., & Lavrač, N. (2019). Py3plex: A library for scalable multilayer network analysis and visualization. In *International Conference on Complex Networks and Their Applications* (pp. 757-768). Springer. https://doi.org/10.1007/978-3-030-05411-3_60

Conference paper focusing on visualization and scalability.

[Blondel2008] Blondel, V. D., Guillaume, J. L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10), P10008. https://doi.org/10.1088/1742-5468/2008/10/P10008

Louvain algorithm for community detection.

[Rosvall2008] Rosvall, M., & Bergstrom, C. T. (2008). Maps of random walks on complex networks reveal community structure. *Proceedings of the National Academy of Sciences*, 105(4), 1118-1123. https://doi.org/10.1073/pnas.0706851105

Infomap algorithm for community detection.

[Mucha2010] Mucha, P. J., Richardson, T., Macon, K., Porter, M. A., & Onnela, J. P. (2010). Community structure in time-dependent, multiscale, and multiplex networks. *Science*, 328(5980), 876-878. https://doi.org/10.1126/science.1184819

Multilayer community detection with modularity optimization.

[Grover2016] Grover, A., & Leskovec, J. (2016). node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 855-864). https://doi.org/10.1145/2939672.2939754

Node2Vec algorithm for network embeddings.

[Perozzi2014] Perozzi, B., Al-Rfou, R., & Skiena, S. (2014). DeepWalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 701-710). https://doi.org/10.1145/2623330.2623732

DeepWalk algorithm for learning node representations.

[Pastor-Satorras2015] Pastor-Satorras, R., Castellano, C., Van Mieghem, P., & Vespignani, A. (2015). Epidemic processes in complex networks. *Reviews of Modern Physics*, 87(3), 925. https://doi.org/10.1103/RevModPhys.87.925

Comprehensive review of epidemic modeling on networks.

[Buldyrev2010] Buldyrev, S. V., Parshani, R., Paul, G., Stanley, H. E., & Havlin, S. (2010). Catastrophic cascade of failures in interdependent networks. *Nature*, 464(7291), 1025-1028. https://doi.org/10.1038/nature08932

Cascading failures in interdependent networks.

[Gligorijevic2019] Gligorijević, V., Barot, M., & Bonneau, R. (2019). deepNF: deep network fusion for protein function prediction. *Bioinformatics*, 35(5), 757-766. https://doi.org/10.1093/bioinformatics/bty440

Application to biological network analysis.

[Magnani2013] Magnani, M., & Rossi, L. (2013). Pareto distance for multilayer network analysis. In *Social Computing, Behavioral-Cultural Modeling and Prediction* (pp. 249-256). Springer. https://doi.org/10.1007/978-3-642-37210-0_27

Social network analysis with multilayer methods.

[Gallotti2014] Gallotti, R., & Barthelemy, M. (2014). Anatomy and efficiency of urban multimodal mobility. *Scientific Reports*, 4(1), 6911. https://doi.org/10.1038/srep06911

Transportation networks as multilayer systems.

[NetworkX] Hagberg, A., Swart, P., & Schult, D. A. (2008). Exploring network structure, dynamics, and function using NetworkX. *Los Alamos National Lab.(LANL), Los Alamos, NM (United States)*.

NetworkX documentation: https://networkx.org/

[Arrow] Apache Arrow Development Team. (2023). Apache Arrow. https://arrow.apache.org/

High-performance columnar data format.